SISTEMAS OPERATIVOS

Aula Teórica 4

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Thread Usage (1)

In many applications, multiple activities are going on at the same time,
decomposing such an application into multiple sequential threads that run in quasi-parallel,
makes the programming model becomes simpler;

Threads = miniprocesses

-Ability for parallel entities to share an address space and all of its data among themselves;

-Since they are lighter then processes, they are easier/faster to create and distroy (10-100 faster than processes);

-Improve performence, speeding up the applications (generaly in I/O bound tasks);

-Are useful in systems with multiple CPU, where real paralelism is possible.

Thread Usage (Example: Word processor with three threads)

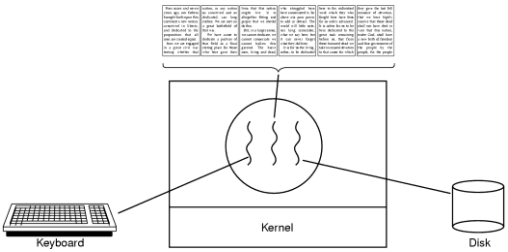
Example: Word processor with three threads

1º thread – interacts with the user
2º thread – handles text reformatting in the background
3º thread – saving monitor, feature that saves the entire file to disk periodicaly

If the program were single-threaded, everything works very slow, with lots of delays between all kind of interactions:
keybord and mouse events,
disk read/writing,
text formatting,
etc.

With three threads the programming model is much simpler, they share a common memory and all of them have access to the document being edited.

With three processes this would be impossible.



Thread Usage (Example: Multithread Web Server)

Example: Multithread Web Server

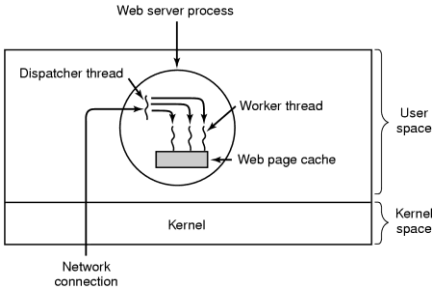
1º thread – dispatcher, reads incoming requests for work from the network
2º thread – worker thread, first idle, it is choosen to hands the request.

To improve performance the Web Server maintains a collection of heavelly used pages in main memory (Cache) to eliminate the need to access the disk.

How it works:
-Dispatcher wakes up the sleeping worker thread (blocked state -> ready state)

-On start, the worker thread checks the Web page cache to see if the request can be satisfied.
- If not, it starts a read operation to get the information from disk (and blocks until the disk operation completes)

-When waiting for the disk operation to complete, possibly the Dispatcher, or another worker thread (ready to run), start running in order to acquire more work.



Thread Usage (Example: Multithread Web Server)

Dispatcher thread

```
while (TRUE) {
    get_next_request(&buf);
    handoff_work(&buf);
}
```

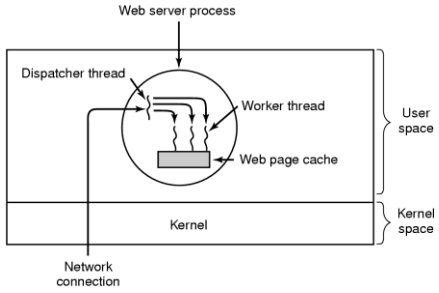
(a)

Worker thread

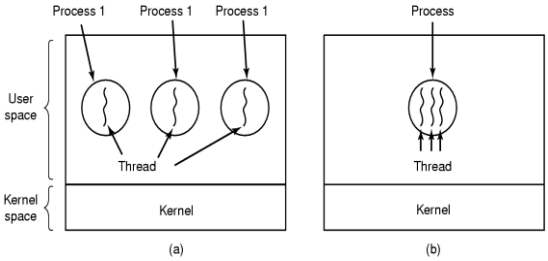
```
while (TRUE) {
    wait_for_work(&buf)
    look_for_page_in_cache(&buf, &page);
    if (page_not_in_cache(&page))
        read_page_from_disk(&buf, &page);
    return_page(&page);
}
```

(b)

Figure 2-9. A rough outline of the code for Fig. 2-8. (a) Dispatcher thread. (b) Worker thread.



Threads - The Classical Thread Model (1)



(a) Three processes each with one thread

(b) One process with three threads

Processes are used to group resources together. They have an address space, with program text and data, as well as resources, like: open files, child processes, pending alarms, signal handlers, accounting information, etc.

Threads are the entities scheduled for execution on the CPU.

Several threads in a process are not as independent as different processes. All threads have exactly the same address space, which means that they also share the same global variables, the same set of open files, child processes, signals, alarms.

There is no protection between threads because:
-it is impossible
-it shoud not be necessary

The Thread Model (2)

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

The first column lists processes properties, shared by all threads in a process.

The second one lists some items private to each thread.

The Thread Model (3)

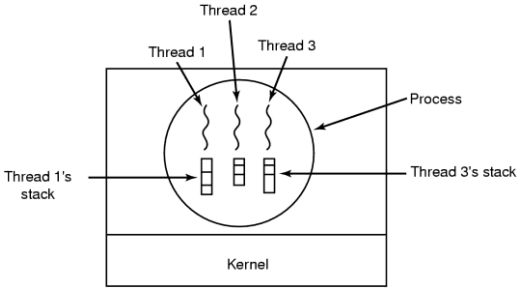
Each thread has its own stack.
Because all of then will generally call different procedures and thus will have a different execution history.

When multithreading is present, processes usually start with a single thread present. This thread has the ability to create new threads by calling a library procedure like: *thread_create*.

When the thread finished its work, it can exit by calling the library procedure *thread_exit*.

One thread can wait for a specific thread to exit by calling the procedure *thread_join*.

Another common thread call is *thread_yield*, which allows a thread to voluntarily give up the CPU to let another thread run.



The Thread Model (POSIX IEEE standard 1003.1c)

While thread are often useful, they also introduce a number of serious complications into programming model.

To start with, consider the effects of the UNIX *fork* system call:
If a parent process has multiple threads, should the child also have them?
If not the process may not function properly, since all of them may be essential. However, if the child gets as many threads as the parent, **what happens if a thread in the parent was blocked on a read call, say, from the keyboard?**
Are two threads now blocked on the keyboard, one in the parent and one in the child?
When a line is typed, do both threads get a copy of it?
Only the parent?
Only the child?

Another class of problems is related to the fact threads share many data structures. **What happens if one thread close a file while another one is still reading from it?**

Thread call	Description
pthread_create	Create a new thread
pthread_exit	Terminate the calling thread
pthread_join	Wait for a specific thread to exit
pthread_yield	Release the CPU to let another thread run
pthread_attr_init	Create and initialize a thread's attribute structure
pthread_attr_destroy	Remove a thread's attribute structure

Figure 2-13. Some of the Pthreads function calls.

The relation between threads and fork can be quite complex.

To make things easier, on a system such as Linux, a *fork* of a multithreaded process will create only a single thread in the child.

Implementing Threads in User Space

There are two main places to implemente threads: the user space and the kernel.

The first method is to put the threads package entirely in user space. The kernel knows nothing about them. First, and most obvious advantage, is that user-level threads package can be implemented on operating system that does not support threads.

Threads are implemente by a library.

Example:
The main program loops NUMBER_OF_THREADS times, creating a new thread on each iteration, after announcing its intention. If the thread creation fails, it prints an error message and then exits. After creating all the threads, the main program exits.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS    10

void *print_hello_world(void *tid)
{
      /* This function prints the thread's identifier and then exits. */
      printf("Hello World. Greetings from thread %d\n", tid);
      pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
      /* The main program creates 10 threads and then exits. */
      pthread_t threads[NUMBER_OF_THREADS];
      int status, i;

      for(i=0; i < NUMBER_OF_THREADS; i++) {
            printf("Main here. Creating thread %d\n", i);
            status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

            if (status != 0) {
                  printf("Oops. pthread_create returned error code %d\n", status);
                  exit(-1);
            }
      }
      exit(NULL);
}
```

Figure 2-14. An example program using threads.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

